

Guide de Préparation à l'Entretien - FleetHub

Ce document a pour but de vous préparer à défendre vos choix techniques et votre architecture lors de l'évaluation du projet FleetHub.

1. Explication de 5 Endpoints (via Swagger UI)

L'interface Swagger UI permet de visualiser et de tester l'API. Voici 5 endpoints clés couvrant les différents rôles de l'application :

1. **POST /api/token/ (Public / Tous rôles)**
 - **Rôle** : Authentification.
 - **Explication** : Ce point d'entrée reçoit les identifiants de l'utilisateur (email et mot de passe). Si les informations sont correctes, il génère et retourne une paire de clés d'authentification : un Access Token (valide pour une courte durée) et un Refresh Token (valide plus longtemps).
 2. **GET /api/vehicules/ (Gestionnaire / Admin)**
 - **Rôle** : Supervision de la flotte.
 - **Explication** : Permet aux gestionnaires de récupérer la liste exhaustive des véhicules. L'API retourne un tableau JSON avec les informations de chaque véhicule (Immatriculation, Statut, Kilométrage, Motorisation). Cela permet d'alimenter le tableau de bord principal.
 3. **POST /api/entretiens/ (Chauffeur)**
 - **Rôle** : Déclaration de panne / Demande d'entretien.
 - **Explication** : Utilisé par un chauffeur pour signaler qu'un véhicule nécessite une révision ou une réparation (qu'elle soit urgente ou non). Le payload JSON inclut le type d'intervention, l'urgence, et lie l'entretien au véhicule actuellement affecté au chauffeur. Le statut initial est `EN_ATTENTE_VALIDATION`.
 4. **PATCH /api/entretiens/{id}/ (Gestionnaire)**
 - **Rôle** : Validation d'un workflow métier.
 - **Explication** : La méthode `PATCH` (mise à jour partielle) est utilisée par le gestionnaire pour approuver ou refuser la demande d'entretien du chauffeur. Par exemple, il met à jour le champ `statut` pour le passer à `PLANIFIE` ou `REFUSE`.
 5. **POST /api/utilisateurs/ (Administrateur)**
 - **Rôle** : Gestion des accès.
 - **Explication** : Réservé strictement à l'Administrateur, cet endpoint permet de créer de nouveaux profils dans le système. L'Admin transmet un objet contenant le nom, l'email, le mot de passe temporaire et le rôle (ex: "CHAUFFEUR").
-

2. Le Token JWT (JSON Web Token) côté Frontend

Le JWT est le standard utilisé par FleetHub pour sécuriser les communications entre React (Frontend) et Django (Backend). Voici son cycle de vie :

- **Récupération** : Lorsque l'utilisateur valide le formulaire de connexion, une requête HTTP `POST` est envoyée à `/api/token/`. Le backend vérifie l'identité et renvoie le token en réponse (généralement `res.data.access`).
 - **Stockage** : Dès réception, le frontend stocke ce token de manière persistante sur le navigateur du client. Dans FleetHub, cela se fait via le `localStorage` (ex: `localStorage.setItem('access_token', token)`). Cela permet à l'utilisateur de rester connecté même s'il actualise la page.
 - **Utilisation** : Pour chaque requête nécessitant d'être connecté (par exemple, afficher la liste des véhicules), le frontend doit prouver son identité. Grâce à un **intercepteur Axios** (configuré dans `api/axios.js`), le token est automatiquement récupéré depuis le `localStorage` et injecté dans l'en-tête (Header) de la requête HTTP sous le format :
`Authorization: Bearer <votre_token_jwt>`
 - **Expiration** : Si l'Access Token expire, l'application peut soit rediriger l'utilisateur vers la page de connexion, soit utiliser le Refresh Token de manière transparente pour demander un nouvel Access Token au backend.
-

3. Choix de Modélisation de la Base de Données

Les choix d'architecture de la base de données relationnelle (modèles Django) ont été guidés par des besoins de traçabilité, de flexibilité et de robustesse :

- **Utilisateur personnalisé (AbstractUser) avec Rôles** :
 - *Choix* : J'ai écrasé le modèle `User` par défaut de Django pour remplacer l'`username` par l'`email` comme identifiant de

connexion.

- *Pourquoi* : C'est plus naturel et professionnel dans un contexte d'entreprise (SSO, réinitialisation de mot de passe). J'ai également intégré un champ "Role" (Admin, Gestionnaire, Chauffeur) directement dans le modèle, ce qui simplifie grandement la gestion des permissions sans avoir à gérer des groupes complexes.

- **La relation entre Véhicules et Chauffeurs (Affectation) :**

- *Choix* : Au lieu de mettre un champ `chauffeur_actuel` directement sur le véhicule (relation One-to-One), j'ai créé une table intermédiaire `Affectation`.
- *Pourquoi* : Cela permet de garder un **historique complet**. L'affectation possède des dates de début/fin et des relevés kilométriques. On peut ainsi savoir exactement "qui conduisait quel véhicule à telle date", essentiel en cas d'infraction routière ou pour évaluer la consommation d'un chauffeur spécifique.

- **L'intégration des Véhicules Électriques (VE) :**

- *Choix* : Le modèle `Vehicule` possède un champ `motorisation` (Thermique vs Électrique). La table `Consommation` accepte la notion de `kwh` à la place des litres, et `Entretien` inclut des types d'interventions propres à l'électrique (ex: Batterie 12V, Haute Tension).
- *Pourquoi* : Pour anticiper l'évolution naturelle des flottes d'entreprise vers l'électrique, le modèle de données ne force pas la présence de champs obsolètes (comme "litres_essence") mais utilise des nomenclatures flexibles (`coût`, `volume_energie`, `type_energie`).

- **Le cycle de vie de l'Entretien :**

- *Choix* : La table `Entretien` n'est pas qu'un simple journal de maintenance, c'est un véritable workflow. Elle contient un champ `statut` (En attente, Planifié, En cours, Terminé, Refusé) et un champ booléen `est_immediat`.
- *Pourquoi* : Modéliser les statuts permet de distinguer une simple alerte préventive remontée par un chauffeur, d'une réparation validée et budgétée par le gestionnaire. La liaison (Foreign Key) avec le `chauffeur` garantit que l'on sait qui a déclenché l'alerte de maintenance.